

UNIT -V

File System

- A file is a named collection of related information that is recorded on secondary storage.
- In general a file is a sequence of bits, bytes, lines or records meaning of which is defined by the creator and user.

FILE ATTRIBUTES:

- **Name.** The symbolic file name is the only information kept in human-readable form.
- **Identifier.** This unique tag, usually a number, identifies the file within the file system; it is the non-human-readable name for the file.
- **Type.** This information is needed for systems that support different types of files.
- **Location.** This information is a pointer to a device and to the location of the file on that device.
- **Size.** The current size of the file (in bytes, words, or blocks) and possibly the maximum allowed size are included in this attribute.
- **Protection.** Access-control information determines who can do reading, writing, executing, and so on.

- ◉ **Time, date, and user identification.** This information may be kept for creation, last modification, and last use. These data can be useful for protection, security, and usage monitoring.

File operations:

1. **Creating a file:**Creating a file means allocating space for it in the filesystem.
2. **Writing a file :**Once a file is created/opened, you can write data to it.
3. **Reading a file:**Reading retrieves stored data from the file.
4. **Repositioning within a file:**Repositioning means moving the file pointer to a specific location (for reading/writing at desired points).
5. **Deleting a file:**To remove a file from the disk, use the `remove()` function
6. **Truncating a file:**Truncating means cutting off all or part of the file's content — making it smaller (or empty)

File Types:

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

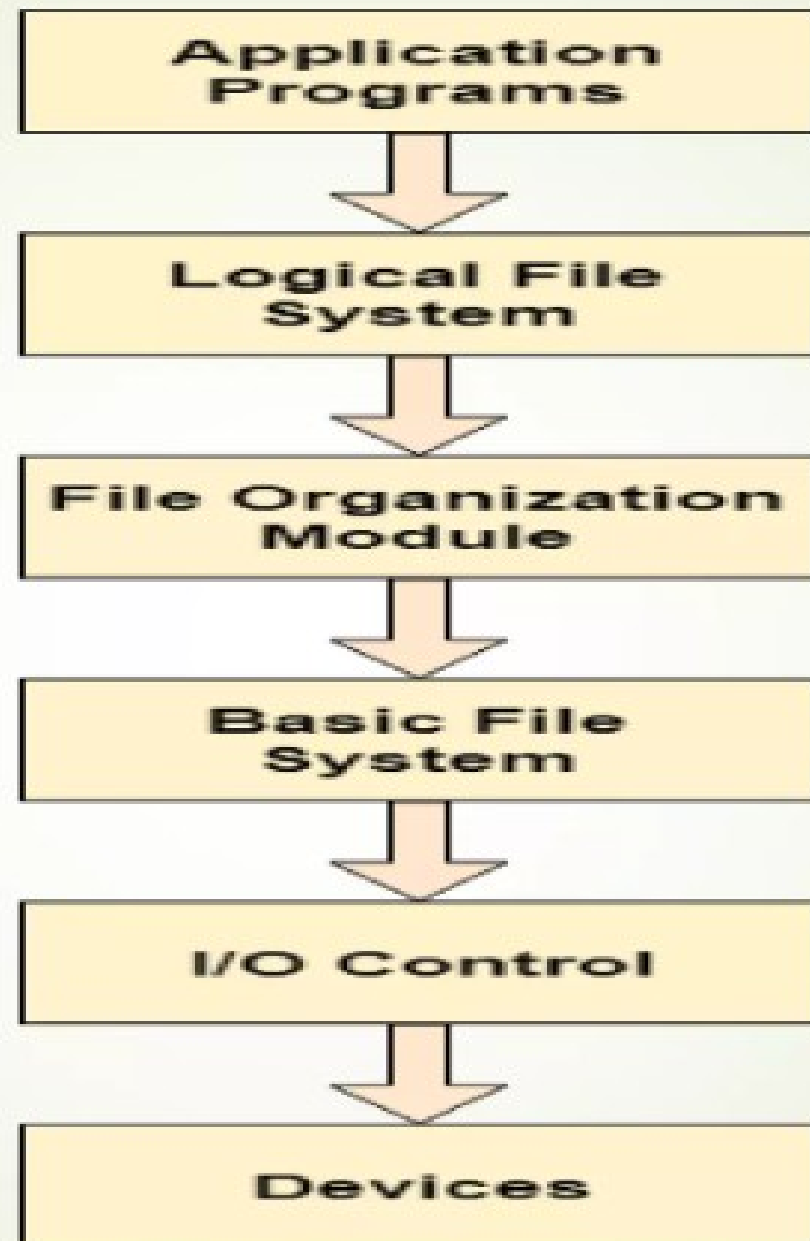
Figure 10.2 Common file types.

File System

- File system is the part of the operating system which is responsible for file management.
- It provides a mechanism to store the data and access to the file contents including data and programs.
- Some Operating systems treats everything as a file for example Ubuntu.
- The File system takes care of the following issues
- **File Structure:** The task of the file system is to maintain an optimal file structure.
- **Recovering Free space:** if file gets deleted its space is recovered
- **disk space assignment to the files :** Using disk scheduling algorithm
- **tracking data location:** We need to keep track of all the blocks on which the part of the files reside.

File System Structure

- File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way.
- A file System must be able to store the file, locate the file and retrieve the file.
- Most of the Operating Systems use layering approach for every task including file systems.
- Every layer of the file system is responsible for some activities.



File system structure

- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure.
- If the application program doesn't have the required permissions of the file then this layer will throw an error. Logical file systems also verify the path to the file.
- Generally, files are divided into various logical blocks.
- Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors.
- Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.

File system structure

- Once File organization module decided which physical block the application program needs, it passes this information to basic file system.
- The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.
- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers.
- I/O controls are also responsible for handling interrupts.

File Access Methods

1. Sequential Access
2. Direct Access
3. Other Access methods

1.Sequential Access:

The simplest access method is *sequential access*. Information in the file is processed in order, one record after the other. This mode of access is by far the most common; for example, editors and compilers usually access files in this fashion.



Figure 10.3 Sequential-access file.

2.Direct Access

sequential access	implementation for direct access
<i>reset</i>	<i>cp</i> = 0;
<i>read next</i>	<i>read cp</i> ; <i>cp</i> = <i>cp</i> + 1;
<i>write next</i>	<i>write cp</i> ; <i>cp</i> = <i>cp</i> + 1;

Figure 10.4 Simulation of sequential access on a direct-access file.

access, and to add an operation *position file to n*, where *n* is the block number. Then, to effect a *read n*, we would *position to n* and then *read next*.

3.Other Access methods

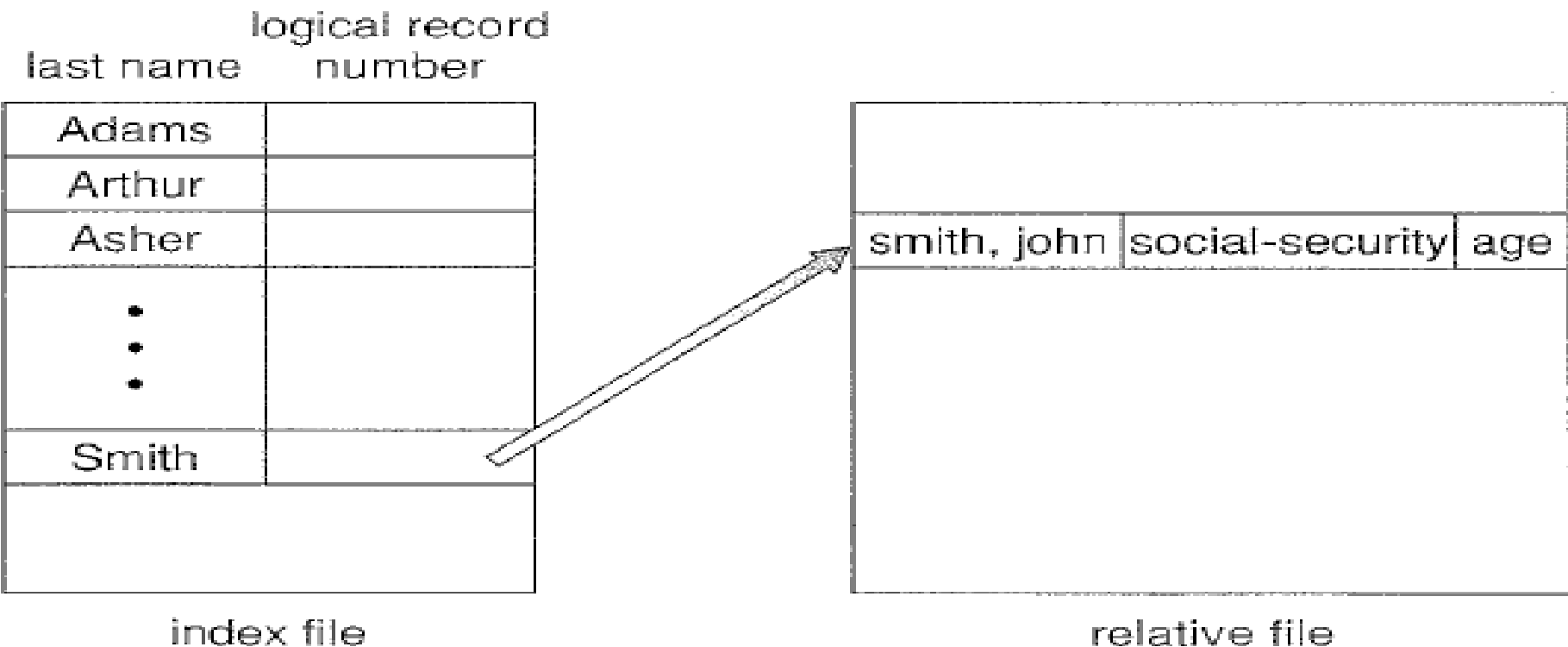


Figure 10.5 Example of index and relative files.

3. Directory Structure

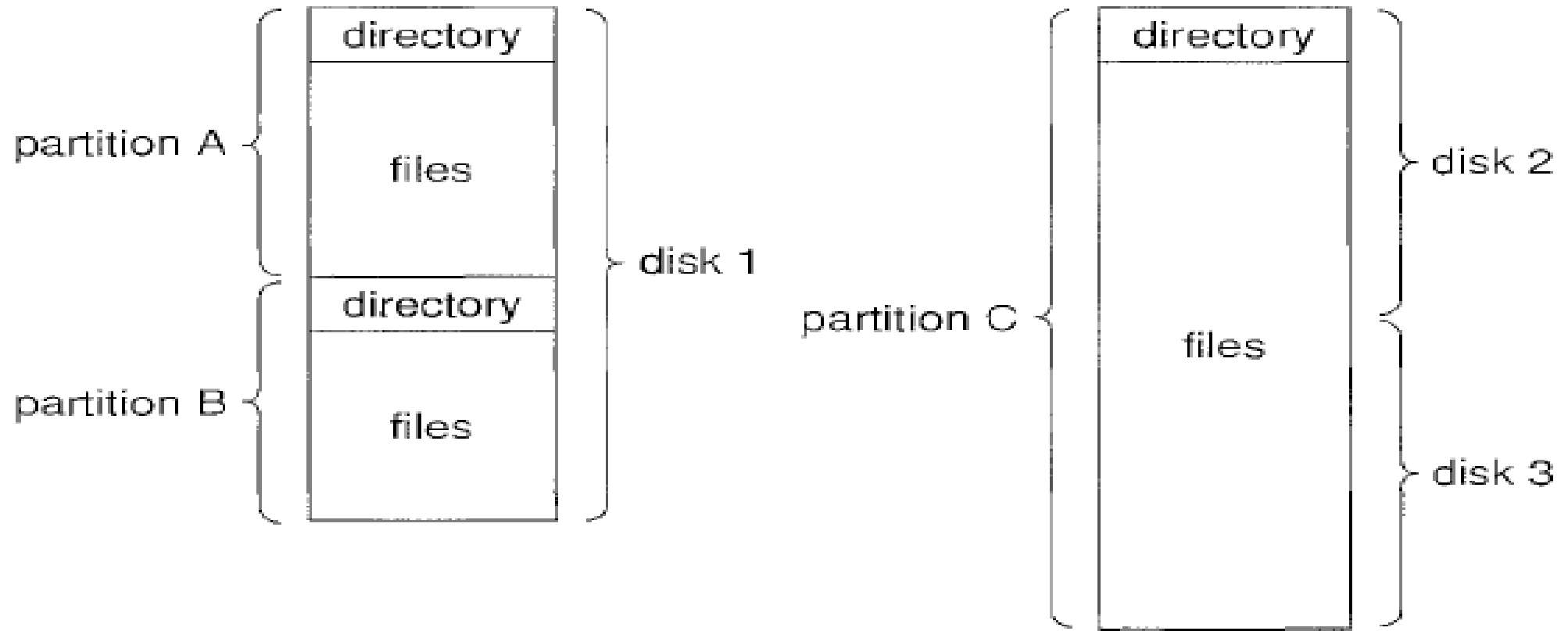


Figure 10.6 A typical file-system organization.

Directory Structures

1. Single Level Directory
2. Two Level Directory
3. Tree Level Directory
4. Acyclic Graph Director

1.Single Level Directory:

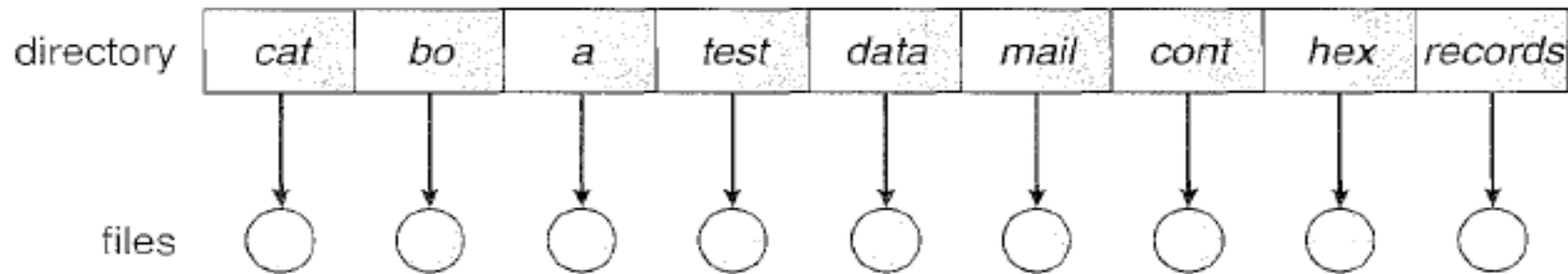


Figure 10.8 Single-level directory.

Two Level Directory

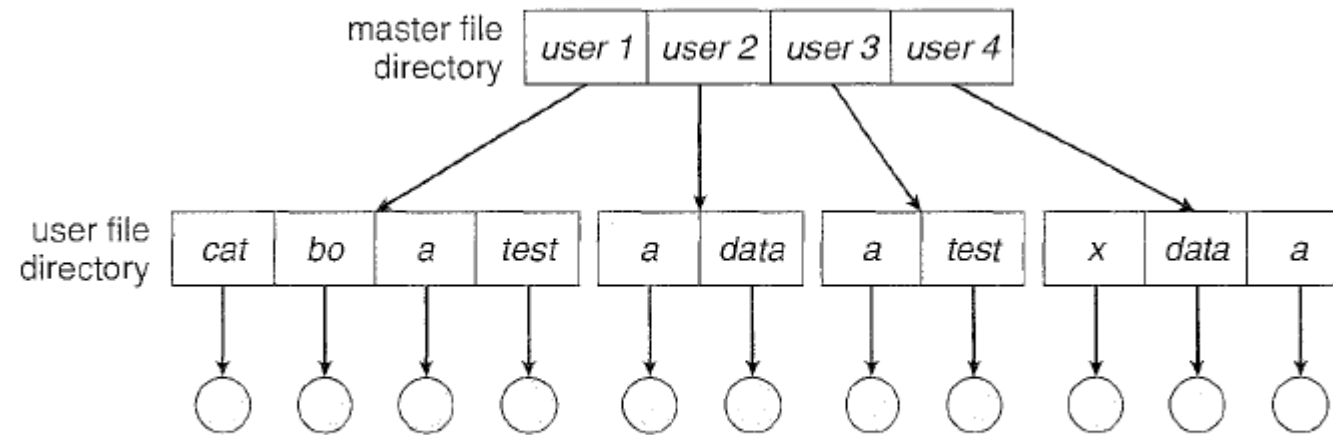


Figure 10.9 Two-level directory structure.

Tree Level Directory

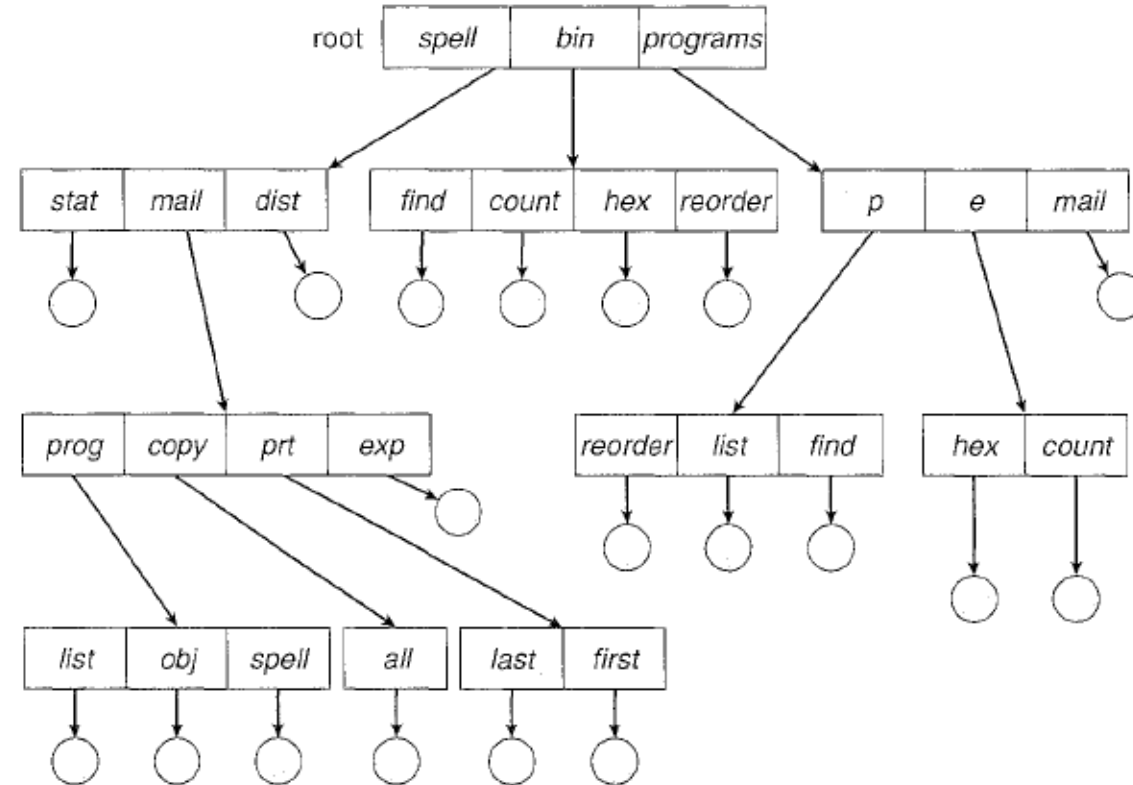


Figure 10.10 Tree-structured directory structure.

Acyclic Graph Director

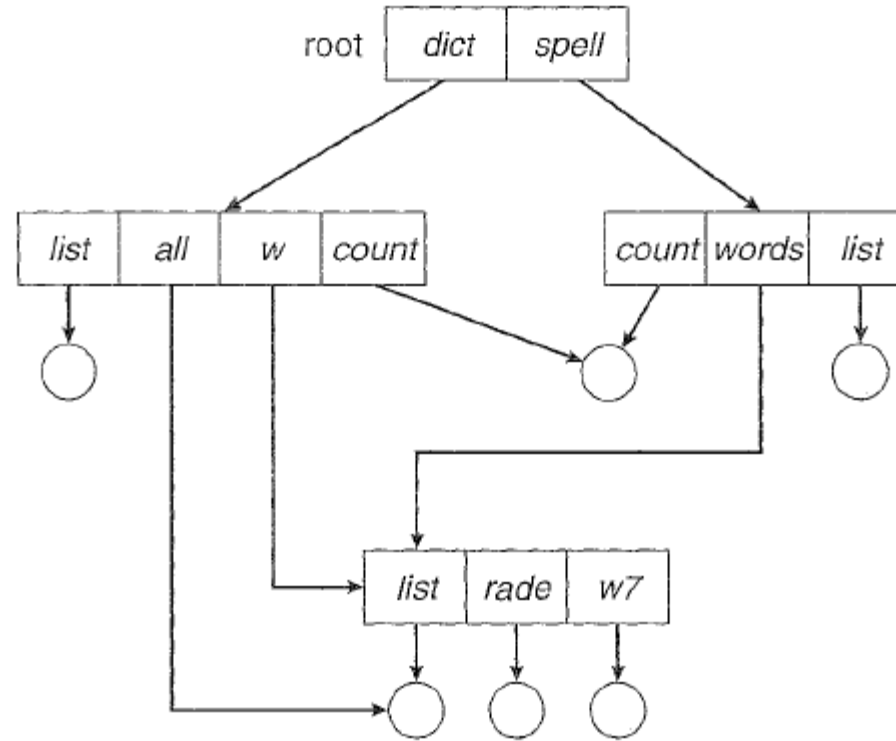
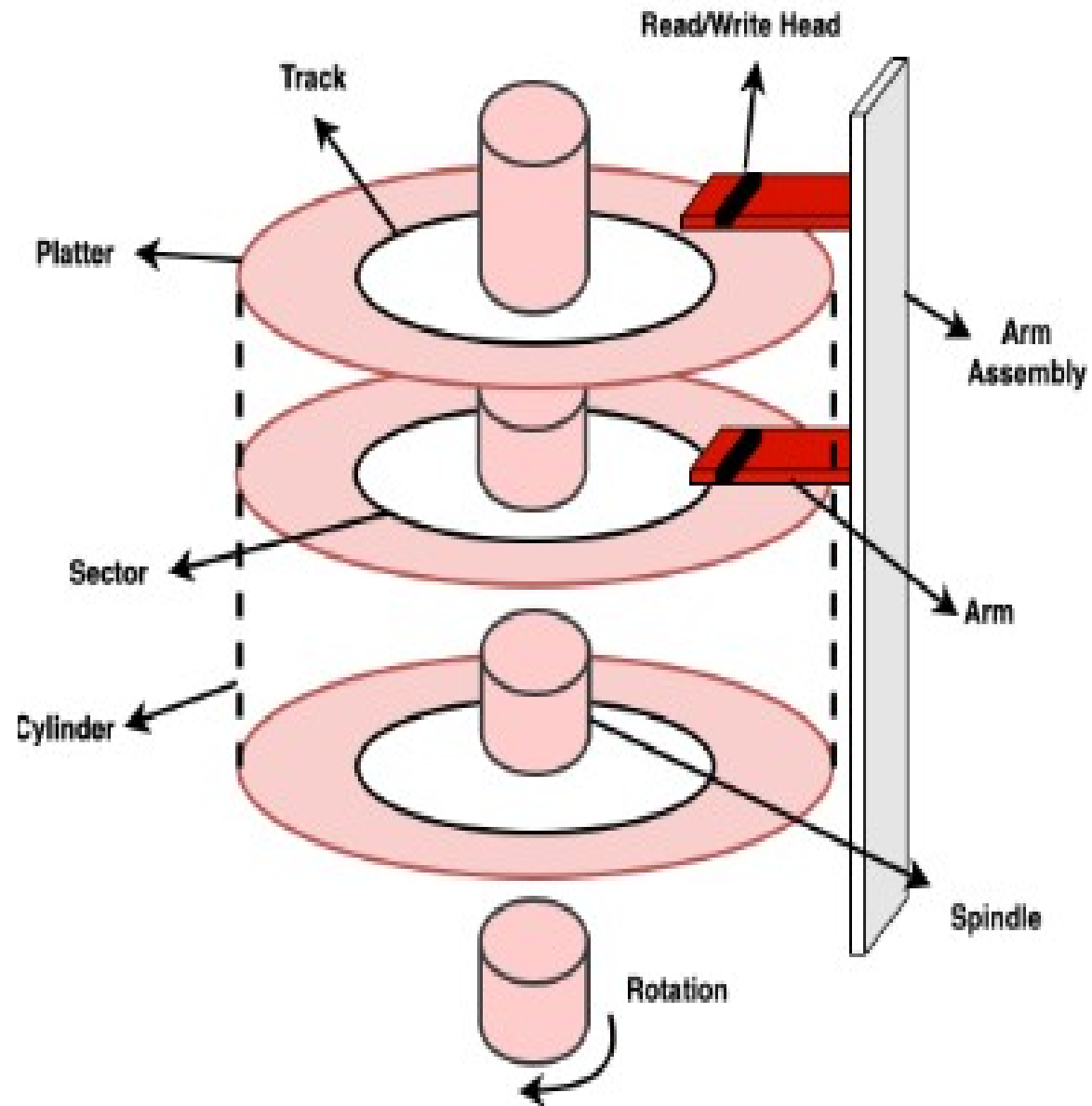


Figure 10.11 Acyclic-graph directory structure.

Disk scheduling Management

- Disk scheduling and management are essential components of a computer's operating system that handle the organization and access of data on a disk.
- Disk scheduling algorithms determine the order in which the read/write head of the disk moves to access data, which impacts the efficiency and speed of accessing data.
- Some commonly used disk scheduling algorithms include First-Come-First-Serve, Shortest Seek Time First, and SCAN.



Disk Scheduling

- A process needs two type of time, CPU time and IO time. For I/O, it requests the Operating system to access the disk.
- At run time, I/O requests for disk tracks come from Process. OS has to choose an order to serve the requests



Important terms in Disk Scheduling

Seek Time

Seek time is the time taken in locating the disk arm to a specified track where the read/write request will be satisfied.

Rotational Latency

It is the time taken by the desired sector to rotate itself to the position from where it can access the R/W heads.

Transfer Time

It is the time taken to transfer the data.

Disk Access Time

Disk access time is given as,

Disk Access Time = Rotational Latency + Seek Time + Transfer Time

Disk Response Time

It is the average of time spent by each request waiting for the IO operation.

Disk Scheduling Algorithms

1. FCFS scheduling algorithm
 2. SSTF (shortest seek time first) algorithm
 3. SCAN scheduling
 4. C-SCAN scheduling
 5. LOOK Scheduling
 6. C-LOOK scheduling
- 
- A decorative curved line in the bottom right corner, transitioning from light blue to light green.

Disk Scheduling Algorithms

- There are several Disk Scheduling Algorithms. We will discuss in detail each one of them.

1. FCFS (First Come First Serve)
2. SSTF (Shortest Seek Time First)
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK

- 1. FCFS (First Come First Serve)
- FCFS is the simplest of all Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue.

1. FCFS scheduling algorithm Example:

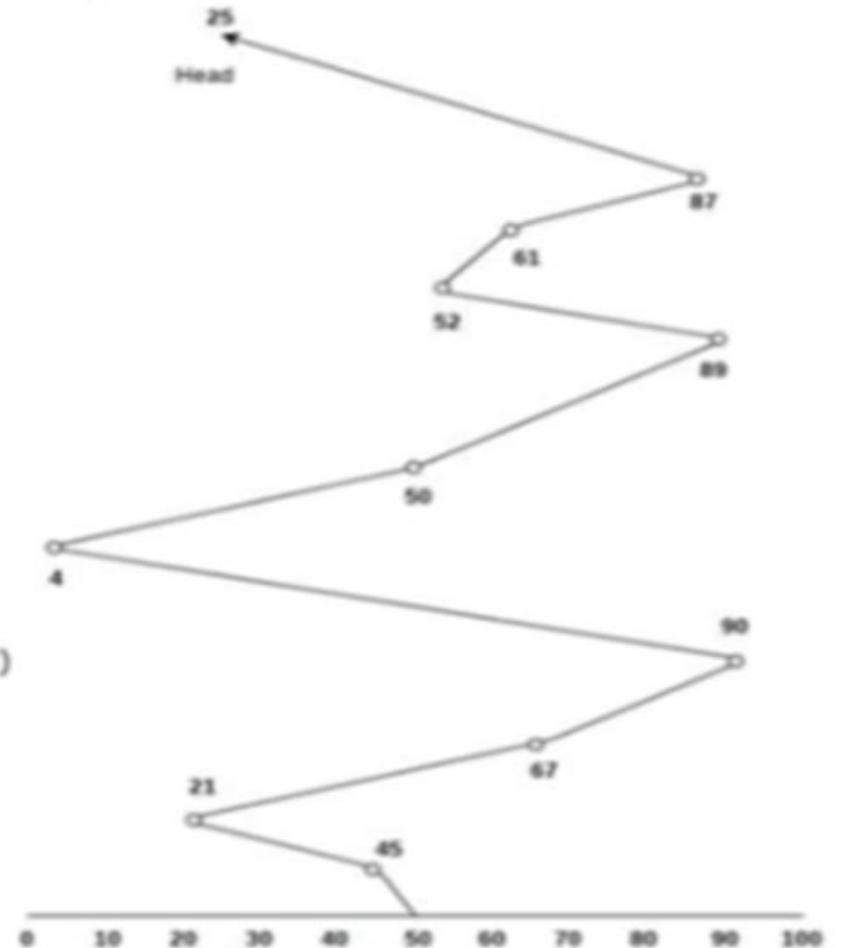
Consider the following disk request sequence for a disk with 100 tracks 45, 21, 67, 90, 4, 50, 89, 52, 61, 87, 25
Head pointer starting at 50 and moving in left direction. Find the number of head movements in cylinders using FCFS scheduling.

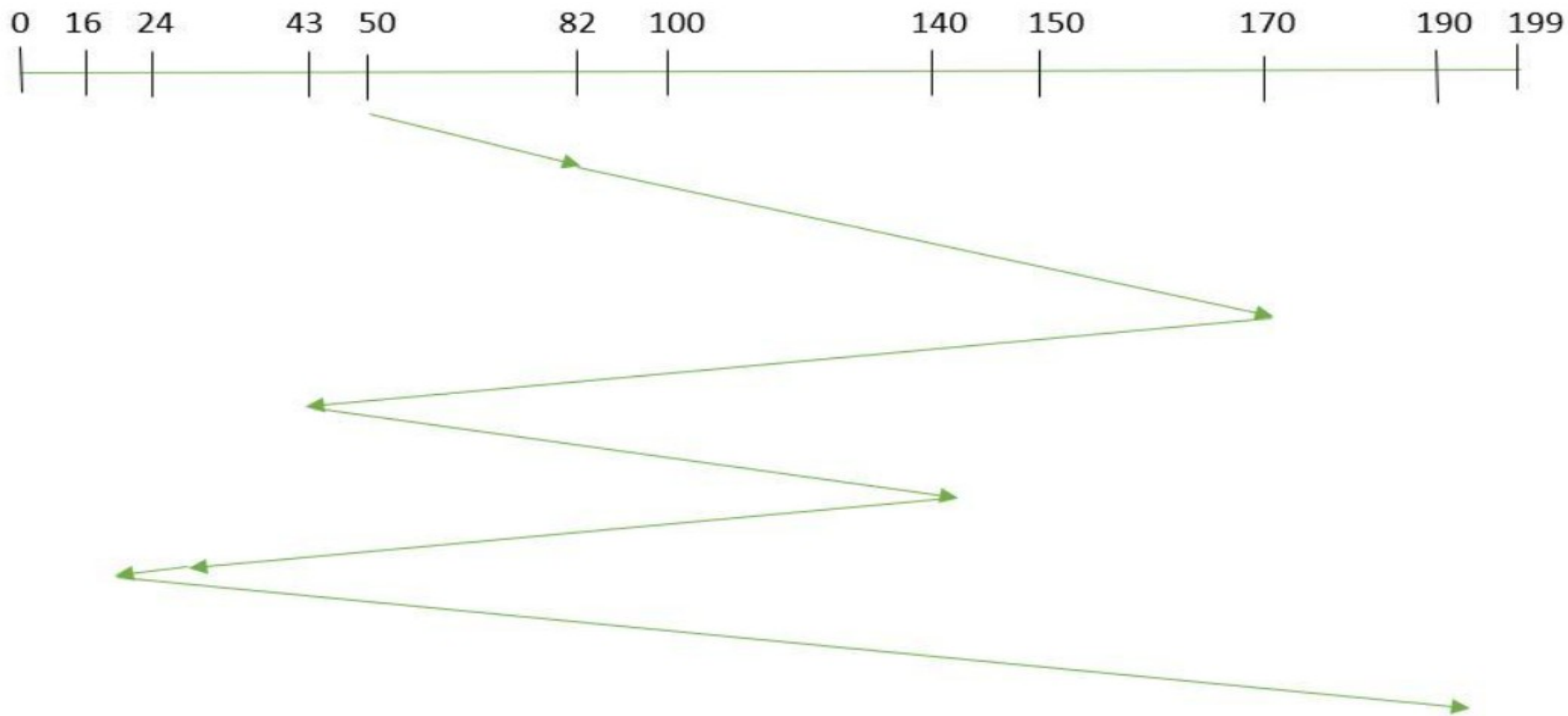
Number of cylinders moved by the head

$$= (50-45) + (45-21) + (67-21) + (90-67) + (90-4) + (50-4) + (89-50) + (61-52) + (87-61) + (87-25)$$

$$= 5 + 24 + 46 + 23 + 86 + 46 + 49 + 9 + 26 + 62$$

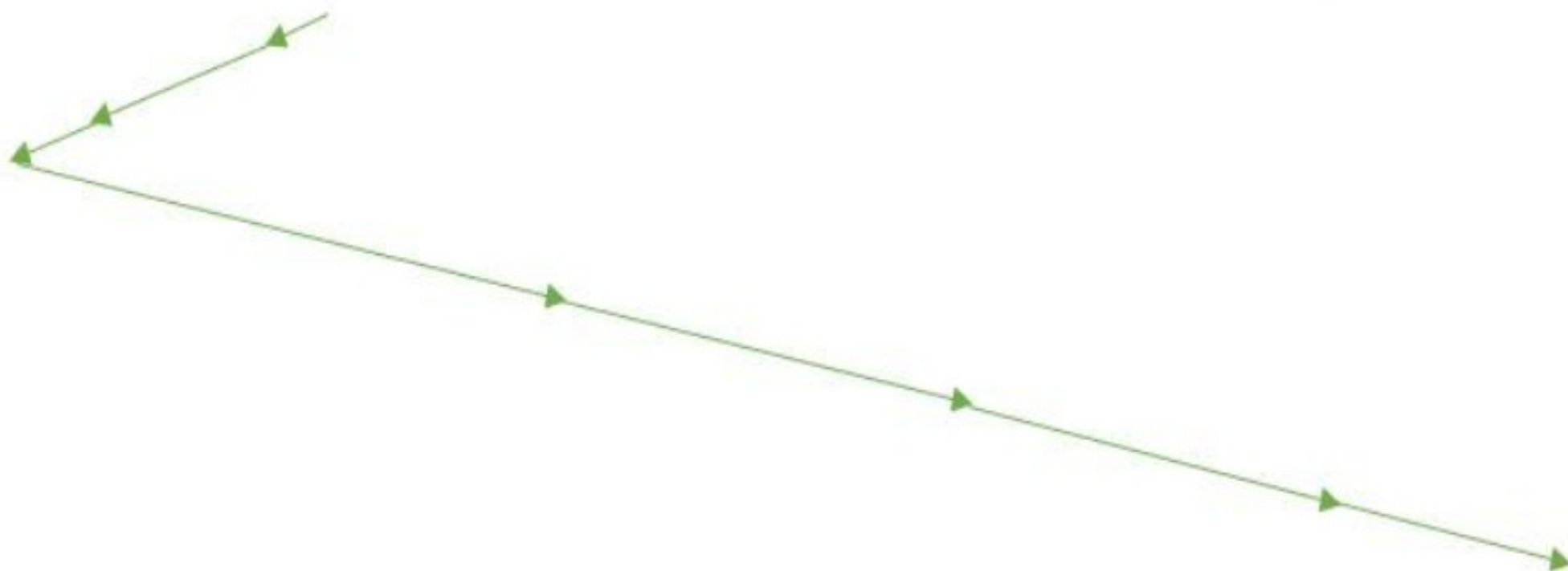
$$= 376$$





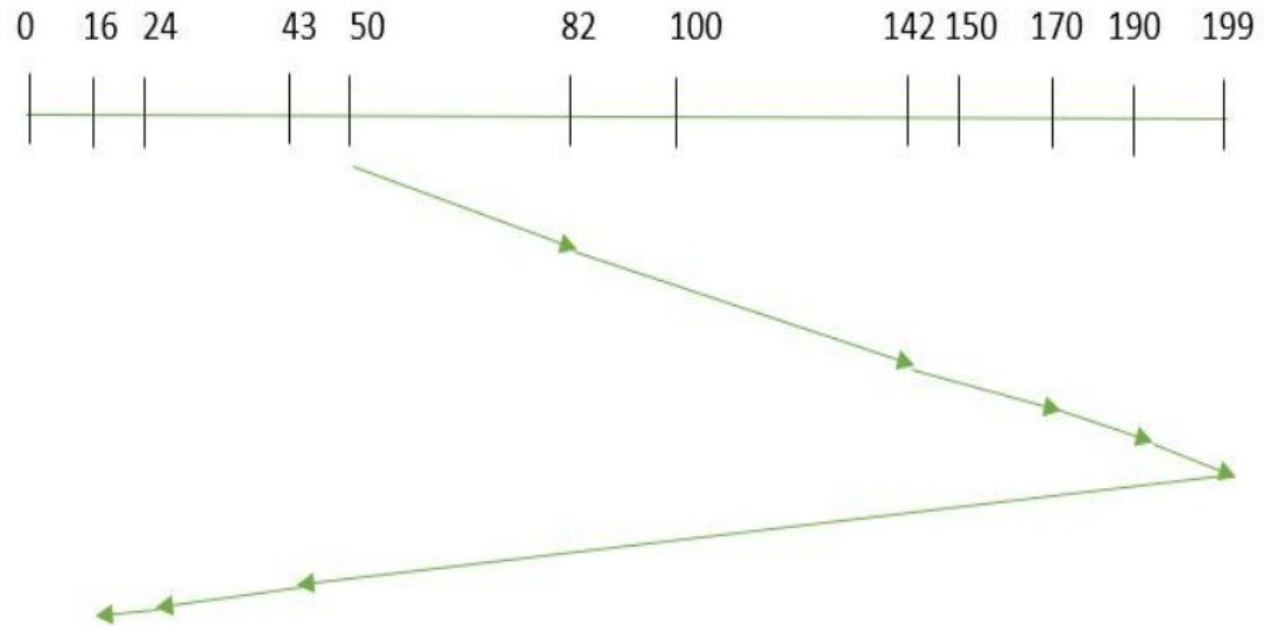
- **2. SSTF (Shortest Seek Time First)**

- In SSTF (Shortest Seek Time First), requests having the shortest seek time are executed first.
- So, the seek time of every request is calculated in advance in the queue and then they are scheduled according to their calculated seek time.
- As a result, the request near the disk arm will get executed first.

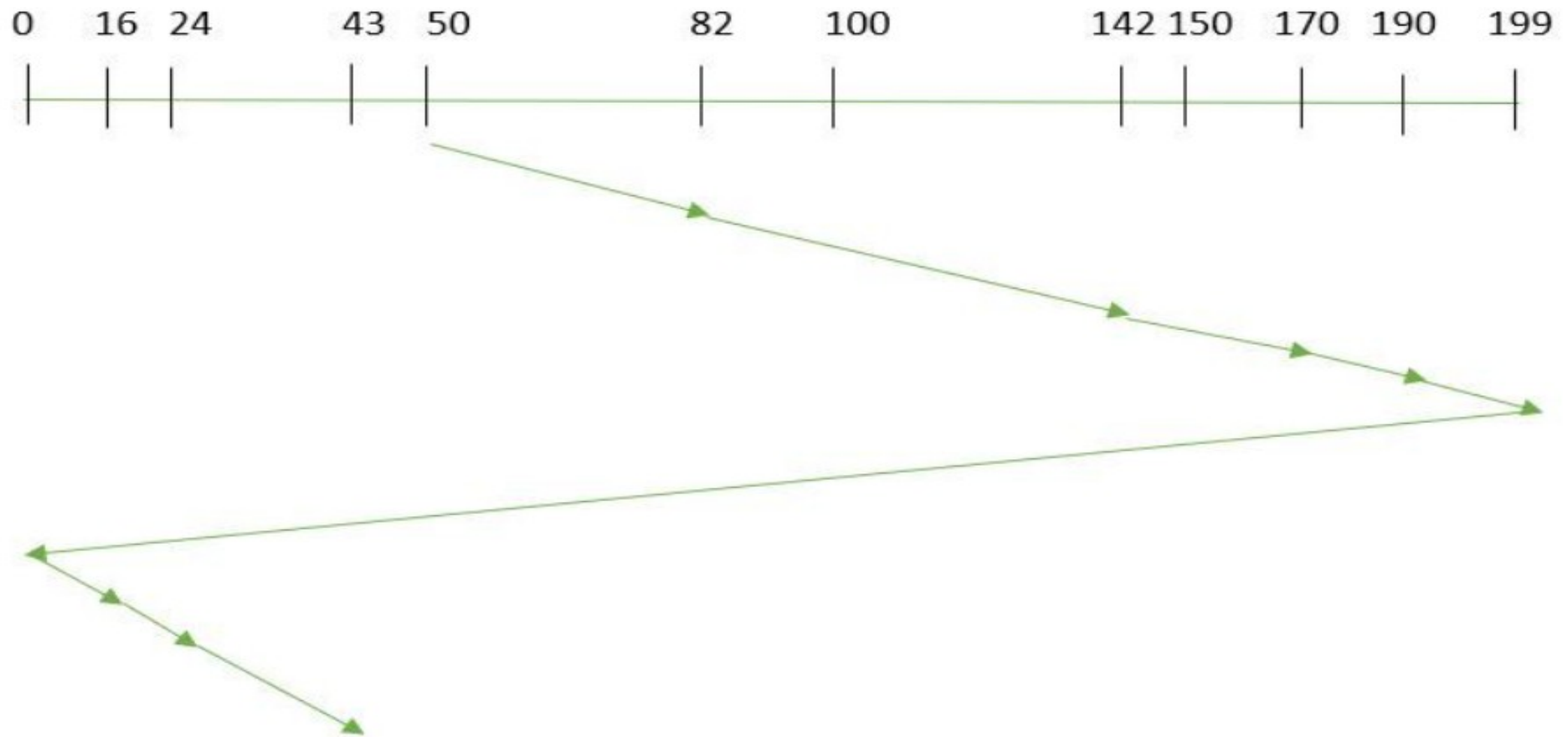


3.SCAN

- In the SCAN algorithm the disk arm moves in a particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path.
- So, this algorithm works as an elevator and is hence also known as an elevator algorithm.

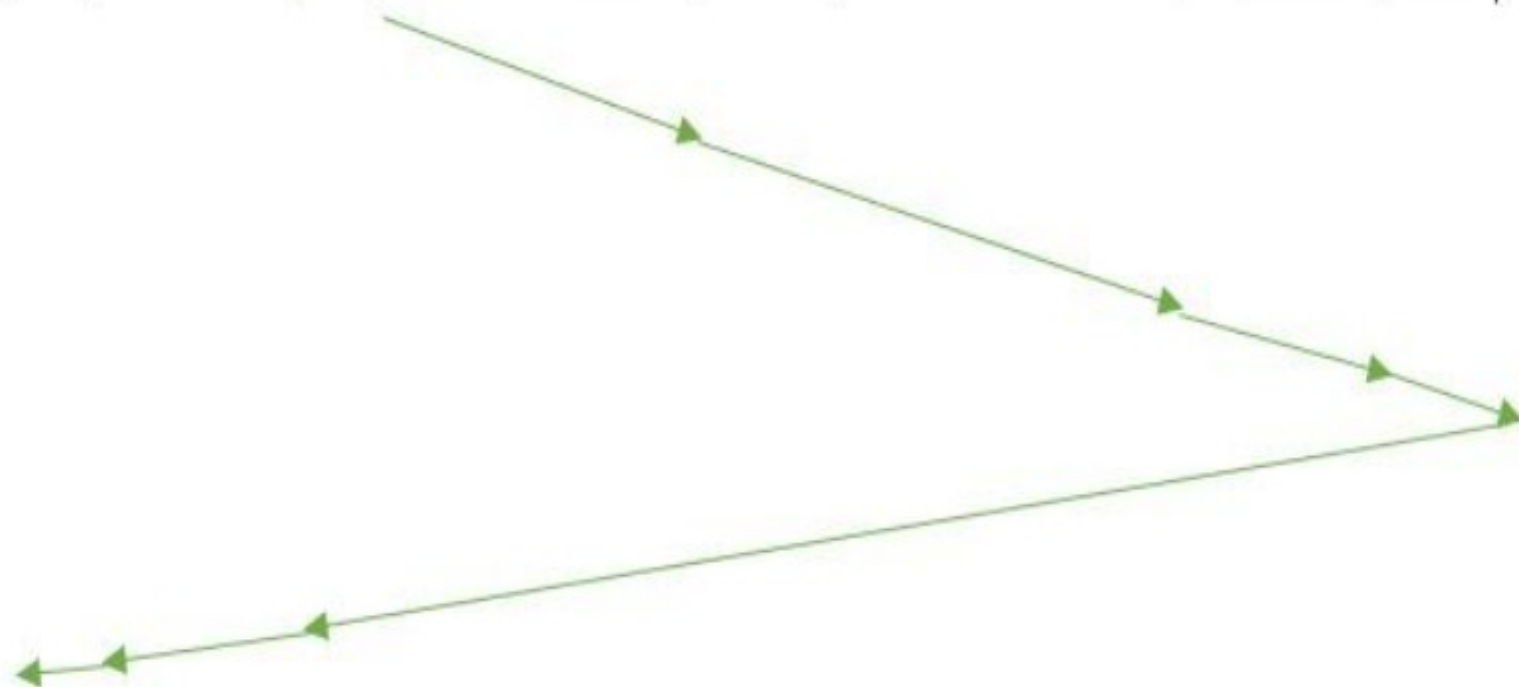
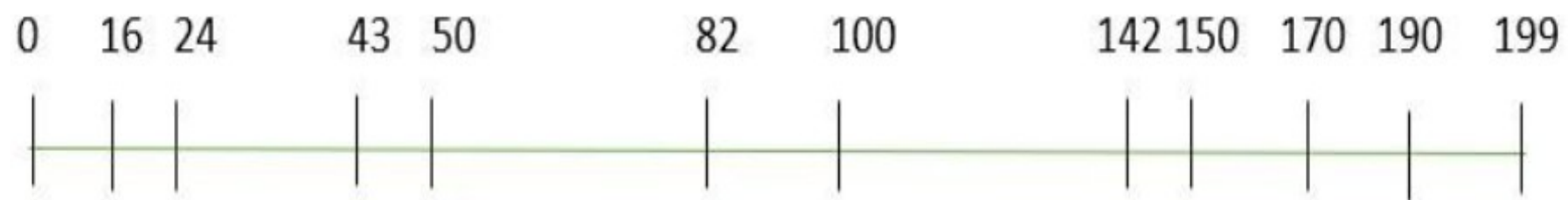


4.C-SCAN



5. LOOK

- LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only.
- Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.



6. C-LOOK

- As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm.
- In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request.
- Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

